Week 13 Milestone Report: Front End and Integration

Course: CS3320 Database Systems, Instructor Dr. Chibuzo Ukegbu
Student: Shritej Gunda (individual project)
Project: Factory Sherlock: Manufacturing Simulation, Quality Control, and Defect Investigation on MySQL
Date: [submission date]
Repository: https://github.com/shritej6/factory-sherlock
Commit or tag: [commit hash after push]

1. Completion status, incomplete items, and blockers

The Week 12 next-step plan listed five items for this milestone. All five are in the repository.

Planned item	Status	Evidence
Usable interface for every module	Complete	Nine soft keys, one per section in frontend/src/app/nav.ts,, each routed in frontend/src/app/router.tsx behind RequireAuth, plus login and the investigation detail route. Screenshots 01 to 09 in docs/media/screenshots.
API contract documented	Complete	docs/api/openapi.json, 35 paths and 44 operations, regenerated at commit c978744; frontend/src/types/api.ts mirrors every schema the interface reads.
Integration tests	Complete	Backend: 481 tests collected by `uv run pytest --collect-only -q` on 2026-11-07, including tests/api/test_status_filters_api.py and tests/api/test_pagination_api.py. Frontend: Vitest 25 files, 118 tests passed the same day.
End-to-end suite	Complete; 9 of 9 passed locally	Playwright, 5 spec files and 9 tests in frontend/e2e; `pnpm test:e2e` passed 9 of 9 on 2026-11-07 at commit 11d5c3a after two navigation locators were repointed at the renamed landmarks; .github/workflows/e2e.yml runs the same suite against a seeded MySQL 8.4 service. [Record the first green End-to-End run id after the push.]
Accessibility pass	Complete	Commit f62170c (skip link, drawer focus, Disclosure focus return, named row actions, heading levels, contrast tokens) with tests in frontend/src/app/AppLayout.test.tsx, frontend/src/components/ui/Disclosure.test.tsx, and frontend/src/features/batches/BatchActions.test.tsx.

Incomplete or carried into Week 14:

- The screenshots were captured on 2026-11-03 and predate the CSV import panel, the control chart, the orders-by-status chart, and the pagination controls; a re-capture with scripts/development/capture_screenshots.py is needed. No video has been recorded.
- docs/testing/test-matrix.md row TM-20 is still a reserved placeholder for the features merged on 2026-11-07, and docs/testing/testing-strategy.md still says no end-to-end test exists.
- The End-to-End and Compose Smoke workflows have not yet run on GitHub because the integration branch has not been pushed. Docker is not installed on the development machine, so the Compose path runs only in CI.

- [List any item that was planned for Week 13 and is not finished.]

Blockers: [describe any blocker, or write "none"].

2. Required technical artifacts and repository commit or tag

Artifact	Location
Usable interface	frontend/src (app, components, features, hooks, services, types, utils)
API contract	docs/api/openapi.json (35 paths, 44 operations), generated by `make export-openapi` (scripts/development/export_openapi.py); mirrored in frontend/src/types/api.ts
Integration and test evidence	backend/tests (api, integration, unit), frontend/src/**/*.test.ts(x), frontend/e2e/*.spec.ts, .github/workflows
Screenshots	docs/media/screenshots/01-login.png to 09-analytics-sql-explorer.png, described in docs/media/screenshots/README.md
Repository commit or tag	[commit hash after push]

Usable interface

Every screen sits behind RequireAuth and the shared AppLayout (skip link, status band, main landmark, soft-key bar). Role handling follows the backend route guards through frontend/src/utils/permissions.ts: viewers read, operators get shop-floor actions, engineers and administrators get everything else.

Screen	Components	What it does	Screenshot
Login	features/login/LoginPage.tsx, hooks/AuthProvider.tsx	Username and password with the backend message inline; a 429 shows the wait from `details.retry_after_seconds`. A 401 or 403 on session restore clears the token; a network error keeps it and offers a retry.	01-login.png
Factory Overview	features/overview/OverviewPage.tsx, MachineStatusCard.tsx, AlertsList.tsx	Machine cards, quality tiles, active order count from `/production-orders/status-counts`, alerts paged ten at a time with acknowledge and the overdue order check, open case count from a filtered one-row page.	02-overview-dashboard.png
Machine Monitoring	features/machines/MachinesPage.tsx, EventTimeline.tsx, MachineEventsPanel.tsx, MachineStatusForm.tsx	Machine list, paged event timeline, change-status form offering only the transitions in utils/transitions.ts, per-machine quality summary.	03-machines-events.png
Production Orders and Batches	features/orders/OrdersPage.tsx, OrderForm.tsx, OrderTransitions.tsx; features/batches/BatchesPage.tsx, BatchForm.tsx, BatchActions.tsx	Paged order table with status filter, create and edit forms, legal transitions only. Batches: order, machine, and status filters applied by the server, create form with material lot lines, complete and quarantine actions named after the batch.	04-orders-and-batches.png (orders screen)

Screen	Components	What it does	Screenshot
Quality Inspection with CSV import	features/quality/InspectionsPage.tsx, InspectionForm.tsx, ImportInspectionsPanel.tsx	Paged inspection table with result and batch filters, record form, defect-rate chart, and a CSV import panel that rejects files over 1 MiB before any request and lists every row error from a 422 with its request id.	05-quality-inspections.png (before the import panel)
Investigations case board	features/investigations/InvestigationsPage.tsx, OpenInvestigationForm.tsx, InvestigationDetailPage.tsx, CaseSummary.tsx, EvidenceCard.tsx, StatusActions.tsx, FlaggedBatches.tsx	List, open form, case summary, one EvidenceCard per rule with its result table and a SqlDisclosure showing the exact SQL, status actions (mark in review, return to open, close with a required conclusion), flagged batches linking into the batch table.	06-investigation-case-board.png
Material Traceability and quarantine	features/traceability/TraceabilityPage.tsx, TraceSearchForm.tsx, QuarantineLotForm.tsx	Forward trace by lot number, backward trace by serial number, quarantine of a lot with a reason; an open trace refreshes after the quarantine.	07-traceability-and-quarantine.png
Analytics with SQL explorer and control chart	features/analytics/AnalyticsPage.tsx, WindowControls.tsx, ControlChartPanel.tsx, SqlExplorer.tsx; components/charts/ControlChart.tsx	Five charts (daily trend, defect rate by machine, pass and fail, defect categories, orders by status), a per-characteristic control chart with stepped limits, and an SQL Explorer that runs every runnable catalogue query with the parameters it binds.	08-analytics-charts.png, 09-analytics-sql-explorer.png (before the control chart and orders-by-status chart)
Simulation	features/simulation/SimulationPage.tsx, FaultScenarioBuilder.tsx	Form bound to `SimulationRequest` with a fault scenario builder and a result summary.	Not captured

Every chart carries a "View data table" disclosure from components/charts/ChartFrame.tsx; every data view has loading, empty, and error states from components/ui/States.tsx.

API contract

The contract is the generated OpenAPI document. scripts/development/export_openapi.py imports the FastAPI application and writes `app.openapi()` to docs/api/openapi.json; the routes and Pydantic schemas are the source and the JSON is never edited by hand. The parts the frontend depends on:

- Error body. Every error response matches `ErrorResponse`: `error`, `message`, `details`, and a required `request_id`. frontend/src/services/http.ts turns any non-2xx response into an `ApiError` with status, error code, details, and request id; a non-JSON body falls back to a generic message and a failed fetch becomes a `NetworkError`.
- Pagination envelope. Six list endpoints (orders, batches, inspections, investigations, machine events, alerts) return `items`, `total`, `limit` (1 to 500, default 50) and `offset` (0 to 1,000,000). `Page<T>` and `PageParams` in frontend/src/types/api.ts mirror this and components/ui/Pagination.tsx reads `total`.
- Status filters. `GET /production-batches` and `GET /investigations` accept a repeatable `status` parameter typed as an array of the matching enum; `buildQuery` in http.ts sends an array as repeated keys. `GET /production-orders` takes a single `status`.

- Status counts. `GET /production-orders/status-counts` returns `OrderStatusCounts`, one integer per order status with every key present. `useOrderStatusCounts` in frontend/src/hooks/queries.ts feeds both the Overview tile and the Analytics chart.
- Batch rows carry `order_number`, so the batch table needs no second lookup of orders, and `GET /analytics/queries` marks each catalogue entry `runnable` and lists its `parameters`, so the SQL Explorer shows only the inputs a query binds.

Integration and test evidence

Layer	Count	What it covers
Backend pytest	481 tests collected (`uv run pytest --collect-only -q`, 2026-11-07) in 35 files under backend/tests	API routes and role guards, error contract, pagination, status filters and counts, CSV import, overdue alerts, login throttle, body limits, rules R1 to R7, state machines, schema boundaries, foreign keys, list ordering indexes, the SQL catalogue against MySQL
Frontend Vitest	25 files, 118 tests passed (`pnpm test`, 2026-11-07)	http client (error envelope, network failure, 401, multipart, repeated query keys), permissions, transitions, chart transforms with per-subgroup limits, Pagination clamping, Disclosure focus, ErrorNotice request id, LoginPage 429 text, role gating and paging on OrdersPage and OverviewPage, AlertsList, ImportInspectionsPanel, TraceabilityPage, SqlExplorer, ControlChartPanel, EvidenceCard, AuthProvider
Playwright	5 spec files, 9 tests in frontend/e2e	See below
GitHub Actions	5 workflows: Backend CI, Frontend CI, Docs Lint, Compose Smoke, End-to-End	Compose Smoke and End-to-End were added this milestone

The Playwright specs drive the real frontend, backend, and MySQL with the seeded demo accounts:

- login.spec.ts (2): a wrong password shows "Invalid username or password" and stays on /login; the engineer lands on the overview, the soft keys list the nine sections of nav.ts in order, and the status band shows the user and role.
- investigation.spec.ts (1): reads the demo day from CNC-03's first batch through the API, opens an investigation through the form, checks the summary names CNC-03 and an elevated defect rate, checks cards R1 to R6 in order, expands the SQL on R1, checks flagged batches, then moves OPEN to IN_REVIEW to CLOSED with a conclusion.
- traceability.spec.ts (3): forward trace of lot L201 lists batches with unit counts; backward trace of a unit from that lot shows its lot and inspection summary; an engineer quarantines an available lot outside CNC-03 and the chip reads Quarantined after a reload.
- roles.spec.ts (2): a viewer sees the orders table with no create, edit, or transition controls; a direct POST with the viewer token returns 403 `AuthorizationError` with no SQL in the message.
- analytics.spec.ts (1): the SQL Explorer runs `batches_requiring_review`, shows a row count and the "batch number" column, and the disclosure shows the query name, the bound threshold, and a SELECT.

frontend/playwright.config.ts starts both servers: `uv run uvicorn app.main:app --port 8013 --no-proxy-headers` from backend/ and `pnpm dev --port 5183` with `VITE_API_PROXY_TARGET` pointed at 8013, leaving the development ports untouched. .github/workflows/e2e.yml starts a MySQL 8.4 service container, writes a CI .env from .env.example, runs scripts/setup/init_databases.sh, `alembic upgrade head`, and `app.seed --demo`, installs Chromium, runs `pnpm test:e2e` with `CI=1`, and uploads the HTML report on failure. .github/workflows/compose-smoke.yml starts the mysql service from docker-compose.yml with `docker compose up --wait`, checks schemas and grants as root inside the container, connects as the application user over TCP, and removes the volume.

3. Decisions, alternatives, assumptions, constraints, and rationale

Decision	Alternative considered	Rationale
Status filters applied by the API (`status` on batches and investigations)	Request the 500-row lookup page and filter in the browser	The list cap is 500 rows, so a client filter would silently miss anything beyond the first page and the displayed total would be wrong. The server applies `WHERE status IN (...)` before pagination, so `total` reflects the filter and the Overview counts open cases from a one-row filtered page.
One `GET /production-orders/status-counts` request	Six single-row list requests, one per status, reading each envelope total	The earlier hook issued one request per order status to build one chart. One GROUP BY returns every count in one round trip with every key present, and the query key shares the `orders` root so any order mutation refreshes it.
Per-subgroup control limits drawn as stepped lines	One constant UCL and LCL computed from the average sample size	Query 10 returns `ucl` and `lcl` per row: grand mean plus or minus three pooled standard deviations over the square root of that subgroup's sample count. A small subgroup gets wider limits and is not flagged against limits meant for a larger one; `toControlChartSeries` judges each point against its own limits and ControlChart.tsx draws them with `type="stepAfter"` around a straight grand mean.
Shared ErrorNotice with the request id for every mutation	Keep InlineNotice with a formatted message string	Only ErrorState showed the request id, so a failed form action could not be matched to a server log line. Components now keep the raw error and hand it to ErrorNotice, which renders the same message and id as ErrorState; the CSV import test asserts the id is shown.
End-to-end sign-in through the API, with one form login under tracing off	Type the demo password into the login form in every spec	A Playwright trace records typed values and request bodies and is uploaded with the CI report, so `signIn` in e2e/support/auth.ts obtains a token with Node's `fetch` outside the browser and stores it under the app's `localStorage` key. login.spec.ts still types the password because the form itself is under test, and runs with `test.use({ trace: 'off' })`.
Pagination clamps back to the last page with rows	Show an empty page past the end, or jump to the first page	After a delete or a narrowed filter the offset can run past the total. Pagination.tsx shows `floor((total - 1) / limit) * limit` and requests it from an effect, so the user stays near where they were; screens also reset the offset when a filter changes.
Explicit focus management in Disclosure and the row actions	Browser default, which drops focus to the document body when the focused element is removed	Disclosure returns focus to its toggle when a form closes itself, and BatchActions moves focus into the reason field and back. Each behaviour has a test.

Assumptions and constraints: the frontend speaks only to `/api/v1`, through the Vite proxy in development; the demo scenario is dated 2026-11-02, so default windows run 2026-11-01 to 2026-11-05; no screen creates products, machines, or material lots. The e2e suite writes one closed investigation and one quarantined lot per run, and `make reset-demo` restores the demo database.

4. AI help: what, why, accepted, revised, rejected, and verification

Tool: Claude Code, used for implementation drafts, review passes over the frontend and the API, and documentation drafts for this milestone. Every change went through eslint, tsc, Vitest, Playwright, and `make check-docs` before commit, and the student read each diff before committing it. The interaction record is in docs/ai_evidence/ai-use-record.md (rows 29 to 43 cover this milestone) and docs/ai_evidence/conversations/06-parallel-feature-integration.md.

[State in your own words which suggestions from this milestone you accepted, which you revised, which you rejected, and how you verified each one.]

5. Lessons learned at this milestone

- In development every browser reaches the API through the Vite proxy from 127.0.0.1, so a login throttle keyed on the client address locked every demo account after five wrong passwords from one person. The address bucket now applies only after a failed verification, and every self-started uvicorn passes `--no-proxy-headers` (commit 18a5903, SECURITY.md).
- Changing six list endpoints from a bare array to the `items, total, limit, offset` envelope broke eight API tests and the e2e helper written against the array shape on parallel branches (commits 7bca7ac and a95df14). A contract change needs a written note to every consumer being built at the same time.
- Two text colours passed a visual check and failed the WCAG 2.1 AA measurement: `ink-faint` on the canvas was 4.42:1 and the request id line on the error background was 3.95:1. Shifting the token from #697280 to #646d7b and moving the request id to `ink-muted` fixed both; the ratios were computed, not estimated.
- The fetch helper always puts a `body` key in the request init, undefined when there is none. The overdue check used to send `{}` to an endpoint that declares no body (commit 1140e0e), and the regression test asserts that `body` is undefined rather than absent.
- [Add one lesson in your own words.]

6. Support or skill development needed for the next stage

[State any instructor support or skill development you need for Week 14, for example help choosing what the walkthrough video must show, or write "None at this time".]

7. Next-step plan (Week 14)

1. Cut a release candidate from the integration branch after the first green run of all five workflows, and record the run ids.

2. Complete docs/testing/test-matrix.md (fill TM-20; add rows for tests/api/test_status_filters_api.py and the five Playwright specs) and correct the end-to-end section of docs/testing/testing-strategy.md.
3. Record defect and fix evidence: the three integration findings in docs/ai_evidence/conversations/06-parallel-feature-integration.md with their commits and tests.
4. Regenerate the final ERD and database/schema/schema.sql and confirm the three Alembic revisions upgrade and downgrade cleanly.
5. Test the README setup path from a clean checkout on the local MySQL route; record the Compose route from the CI log.
6. Re-capture the nine screenshots with the CSV import panel, the control chart, and pagination; record the walkthrough video and link it from docs/media.
7. Draft the final report and assemble the AI evidence package (ai-use-record.md, the conversation files, the student annotations).

8. Personal reflection (Shritej Gunda)

[Write 150 to 250 words in your own voice. Replace each prompt.]

[Which screen or interaction was hardest to get right, and how did you confirm it works against the real API rather than a mock?]

[Explain one of the Section 3 decisions in your own words as you would in the defense, for example why status filters live in the API or why the control limits step per subgroup.]

[Describe one AI suggestion from this milestone that you revised or rejected, how you found the problem, and what you learned from it.]

[What do you need to practice before the Week 14 testing and readiness defense?]

9. Personal-work summary

[List the artifacts you personally produced or verified for this milestone, one line each, with the file path or screen name.]